

AtMem Doctor

Market Entry and Traction Plan

Working thesis: diagnose memory failures first, then convert successful diagnosis into AtMem governance.

Entry product	AtMem Doctor
First customer	Developers and technical operators running persistent agents that already use memory
Success condition	Users return to AtMem when a real memory failure occurs, then adopt AtMem to prevent repeats

18 September 2026

1. The angle to explore

Do not enter the market by asking users to replace their current memory system. Enter by helping them diagnose a problem in the memory they already have.

Positioning: AtMem Doctor tells a developer whether an agent is remembering the right things, where the problem came from, and what action would correct it.

Doctor	Detect and explain memory failures
Fix	Apply or recommend corrections in the current memory stack
AtMem	Continuously govern memory so the same failure is less likely to return

The commercial sequence is therefore: diagnose, prove the problem with evidence, fix the immediate issue, then offer AtMem as the continuous control layer.

2. First target market

Start with people who already operate agents with persistent memory. The first segment should have enough usage history for memory failures to appear and enough technical ability to inspect a diagnosis.

- Developers running personal or team agents for days or weeks rather than one session.
- Operators using OpenClaw, Mem0, Zep, Graphiti, a vector store, or a custom memory service.
- Teams where a wrong remembered fact can waste debugging time or change agent behaviour.
- Users already reporting stale recall, contradictions, forgotten facts, duplicate memory, unexplained retrieval, or deleted information returning.

Do not target first

- People building simple stateless chatbots.
- Beginners who have not yet experienced a memory failure.
- Large enterprise buyers who need procurement, compliance review, and many integrations before a test.
- Users who mainly want a larger context window rather than persistent memory.

3. The problem signals to hunt for

Do not ask the market a vague question such as whether agent memory is important. Find people who have already described a concrete failure.

Signal	What the user experiences
Wrong recall	The agent retrieves a memory that is no longer correct.
Contradiction	Two active memories describe incompatible facts or preferences.
Stale memory	Old information remains eligible for recall after newer information exists.
Duplicate memory	The same fact is stored several times and distorts ranking or context size.
Memory loss	Information expected to persist disappears after

	restart, migration, upgrade, or compaction.
Ghost memory	Deleted or supposedly forgotten information still influences retrieval.
Unknown source	A developer cannot determine where a remembered fact came from.
Unexpected injection	A memory enters the prompt even though the developer did not expect it.

4. Where to find the first users

Use public evidence of pain as the lead list. The goal is not broad awareness. The goal is to find twenty people who have already spent time debugging memory.

1. Search issue trackers for memory failures in agent and memory repositories. Use terms such as stale memory, wrong memory, duplicate memory, lost memory, memory search, contradiction, forgot, deleted memory, context injection, migration, and retrieval.
2. Check repository discussions and community channels for users posting reproducible failures rather than general feature requests.
3. Create a list of thirty prospects. Record the system they use, the failure described, whether it is reproducible, and whether the issue is still open.
4. Contact the person around the failure they reported. Do not start with a product pitch.
5. Offer a read only diagnosis of their current memory data or a local diagnostic session.

Suggested first contact

I saw the memory issue you reported. I am testing a diagnostic tool for persistent agent memory. The goal is to identify stale, conflicting, duplicate, or unexplained memories without asking you to replace your current stack. If the issue is still happening, I would like to run a read only diagnosis and show you what it finds.

5. The first offer

Make the first offer small enough that a developer can say yes without architecture work. AtMem Doctor should initially be read only and local where possible.

- Input: export, API access, or adapter to the existing memory store.
- Analysis: inspect active memories, recency, duplicates, contradictions, provenance, reachability, and retrieval behaviour.
- Output: a report that names each suspected issue and shows the evidence.
- Action: recommend a correction. Apply a correction only when the user explicitly chooses it.
- Follow up: ask whether the identified issue explains an observed agent failure.

Example diagnostic output

Memory system: existing store
Memories scanned: 4,218

Potential issues	
Conflicts	31
Likely duplicates	74
Potentially stale	19
Missing provenance	46
Unexpected retrievals	8

Top issue
Memory 182 and Memory 411 conflict.
Memory 411 is newer.
Both remain eligible for recall.

Suggested action
Supersede Memory 182 or remove it from retrieval.

6. What AtMem Doctor must prove

The Doctor is useful only if it reduces uncertainty during a real failure. Measure behaviour, not compliments.

Metric	Question
Problem incidence	How many diagnostic sessions find at least one issue the user considers real?
Diagnosis usefulness	Did the report identify the cause of an observed agent behaviour?
Time to diagnosis	How long from connection to a useful explanation?
Fix acceptance	How often does the user act on the recommendation?
Repeat usage	Does the user run Doctor again when another problem appears?
Conversion signal	Does the user ask for continuous monitoring or prevention?

7. Four week traction sprint

Week 1: find pain

- Create a prospect sheet with thirty public memory failure reports.
- Classify each failure into a small taxonomy.
- Contact at least fifteen people.
- Book five diagnostic conversations.
- Do not add major product features this week.

Week 2: manual diagnosis

- Run at least five diagnostic sessions, even if some analysis is manual.
- Record the exact data needed to explain each failure.
- Record which findings the user considers useful and which they ignore.
- Identify the two most common failure classes.

Week 3: automate the repeated work

- Build only the checks that appeared repeatedly in Week 2.
- Choose one ecosystem or storage format for the first adapter.
- Keep the tool read only by default.
- Generate an evidence report a developer can understand without you present.

Week 4: test repeat use

- Put the tool in the hands of ten users.
- Ask them to run it without a live session with you.
- Track whether they return after another memory problem.
- Ask three users to keep AtMem connected as a continuous governance layer.
- Use observed failures to set the next roadmap item.

8. Traction gates

Use explicit gates so you can tell whether the wedge is working.

Gate	Initial test threshold
Lead quality	At least 10 of 30 prospects confirm the memory problem is still relevant.
Diagnostic value	At least 5 users receive a finding they agree is a real issue.
Action	At least 3 users apply or ask to apply a recommended correction.
Repeat behaviour	At least 3 users run Doctor again or ask for continuous monitoring.
Product pull	At least 2 users ask for prevention, monitoring, or integration rather than a one time report.

How to interpret the result

- Doctor finds issues and users act on them: continue with this segment.
- Doctor finds issues but users do nothing: the failures may be low cost or the diagnosis may not be trusted.
- Users care but every integration is different: narrow to one ecosystem until the workflow is repeatable.
- Users repeatedly ask for the same preventive control: that control becomes the next AtMem feature.
- Few target users recognise the problem: change the segment before adding more product.

9. Product ladder

The product can expand only after the diagnostic wedge proves demand.

Stage	Job
AtMem Doctor	Read only diagnosis. Detect, explain, and produce evidence.
AtMem Fix	User approved correction through the existing memory system.
AtMem Protect	Continuous checks for stale, conflicting, duplicate, or unsafe memory.
AtMem	Full governed memory layer with provenance, supersession, quarantine, forgetting, and investigation.

10. What not to build yet

- A large dashboard before users repeatedly ask for one.
- Feature parity with every memory database.
- Many adapters before one target ecosystem proves useful.
- A generic memory health score with no evidence behind the findings.
- Automatic destructive fixes without explicit approval.
- Enterprise controls before the diagnostic workflow earns regular usage.

11. The next ten actions

6. Write the one sentence promise: AtMem Doctor finds why a persistent agent is remembering the wrong thing.
7. Define six initial failure classes: stale, conflict, duplicate, lost, unexplained source, unexpected retrieval.
8. Create a prospect sheet with thirty real memory failure reports.
9. Choose the first ecosystem where you can inspect memory with the least integration work.
10. Contact fifteen users around the specific failure they reported.
11. Run five manual or assisted diagnostic sessions.
12. For every session, record the symptom, root cause, evidence, recommendation, and whether the user acted.
13. Automate only the two checks that repeat across sessions.
14. Give ten users a self serve read only Doctor and watch for repeat use.
15. If at least three users return or ask for continuous monitoring, build the transition from Doctor into AtMem Protect.

12. Core hypothesis to test

Developers operating persistent agents will adopt a diagnostic tool before they adopt a replacement memory layer, and a useful diagnosis can create the trust required for them to enable AtMem governance.

If this hypothesis is wrong, the Doctor experiment should reveal it quickly. That is useful. It prevents the roadmap from expanding around features that have not produced user pull.